Решение задания-кейса II этапа олимпиады «Россети» за 8 класс

Емельяненко Владимир Фёдорович

23 марта 2026 г.

Аннотация

Эта небольшая статья описывает мои решения для задачи-кейса олимпиады «Россети» за 8 класс — стратегии по оптимальному размещению линий электропередач.

Использование ИИ: из-за большого распространения ИИ, я за то, чтобы регламентировать его использование (это уже используется на Национальной Технологической Олимпиаде, откуда я и взял такую привычку).

- Для написания кода мне помогал ИИ-ассистент `Github Copilot`, но подмечу, что это был не простой копипастинг, а лишь помощь в рефакторинге кода.
- Я *не* подсматривал у ИИ решения задачи, которые будут описаны здесь, и решение, которое отправлено Вам как мой окончательный ответ. Эта статья написана на `latex` вручную с помощью `Overleaf`, также без его участия.

Полезные ссылки:

- Репозиторий `GitHub`: `https://github.com/VEFrenovator/rn-2026`
- Интерактивный сайт: `https://rn-2026.onrender.com/`
- Исходный код этой статьи на `latex`: `https://www.overleaf.com/read/grfyjxsntyyg#4c6553`

Содержание

1 Интерактивная визуализация, кодинг и математика **2**

1.1 Представление элементов сети как классов . . . 2

1.1.1 `Node` — класс потребителей (вершин графа) . . . 2

1.1.2 `Connection` — класс линий электропередач (ребёр) . . . 3

1.1.3 `Equipment` — класс оборудования на соединениях . . . 4

1.1.4 `Network` — класс сети . . . 4

1.2 Рассчёт стоимости сети . . . 5

1.2.1 Рассчёт затрат на строительство (B) . . . 5

1.2.2 Рассчёт убытка от недоотпуска электроэнергии (H) . . . 6

1.2.3 Рассчёт стоимости всей сети (C) . . . 6

1.3 Интерактив — реализация и инструкция по использованию . . . 7

2 Стратегии оптимальных соединений **8**

2.1 Централизованная структура вокруг наибольших мощностей . . . 8

2.2 Древовидная структура с единичным корнем в $(12.5, 0)$ . . . 8

3 Заключение **8**

1 Интерактивная визуализация, кодинг и математика

После прочтения условий кейса и анализа распределения потребителей на плоскости самый первый шаг, который хочется преодолеть перед тем, как начинать размышлять о возможных структурах — реализовать способ *визуализировать* а) положения потребителей и б) создаваемые соединения между ними. Хочется «поиграться» с расстановкой и соединениями. Карандаш и линейка отлично подходят для нас — людей, — но в стремлении к автоматизации безусловно хочется иметь инструмент автоматического расчёта. Хорошим вариантом будет реализации `Dash`-веб-приложения на `Python`, т.к. эта библиотека уже поддерживает интерактивность. Забегая вперёд, признаюсь: придётся жертвовать многими хорошими идеями оптимизации в пользу сохранения автоматического расчёта.

Однако, для визуализации необходима целая экосистема для хранения информации о текущем состоянии сети. Более всего, на мой взгляд, здесь подойдёт объектно-ориентированное программирование: представив точки потребителей (вершины графа), линии электропередач (рёбра графа), оборудование на линиях и всю сети в целом как классы, можно будет легко создать логику интерактива.

1.1 Представление элементов сети как классов

1.1.1 `Node` — класс потребителей (вершин графа)

Здесь нет никаких сложностей, нам нужно лишь запоминать координаты вершины, а также рёбра, из неё исходящие.

```python
class Node:
    def __init__(
        self, coords: Tuple[int, int], connections: List["Connection"] = None
    ) -> None:
        self.connections = connections if connections is not None else []
        self._coords = coords
        self.id = str(uuid())

    @property
    def coords(self) -> Tuple[int, int]:
        return self._coords

    @coords.setter
    def coords(self, value: Tuple[int, int]) -> None:
        raise TypeError(
            f"""Попытка изменить координаты нода,
            который не является ConsumerNode.
            Истинный класс нода {type(self)}."""
        )

    def __eq__(self, other: "Node") -> bool:
        return self.id == other.id

    def __hash__(self) -> int:
        return hash(self.id)
```

Напомню, что это лишь общий класс. В задаче можно выделить 3 различных вида вершин: потребителей, подстанции и точки спайки. Заметим также, что у класса потребителей нужно отдельно реализовать запоминание мощности данного потребителя, а у подстанции — возможность изменять координаты. В отличие от этого, у точек спайки нет никаких особенностей в этом плане, и их класс будет просто копировать общий.

```python
class ConsumerNode(Node):
    def __init__(self, power: int, **kwargs) -> None:
```

```
        super().__init__(**kwargs)
        self.power = power


class SolderingNode(Node):
    pass


class SubstationNode(Node):
    def __init__(self, coords: Tuple[int, int] = (0, 0)) -> None:
        if 0 not in coords:
            raise ValueError("Координаты подстанции {coords} не содержат 0.")
        super().__init__(coords)

    @Node.coords.setter
    def coords(self, value: Tuple[int, int]) -> None:
        if 0 not in value:
            raise ValueError("Координаты подстанции {coords} не содержат 0.")
        self._coords = value
```

Как вы видите, в классе вершины для подстанции мы переопределяем установщик координат. Это связано с тем, что только у подстанции можно изменять координаты.

1.1.2 `Connection` — класс линий электропередач (ребёр)

Логика этого класса будет немного сложнее. Нам необходимо реализоватьж:

1. Запоминание двух точек начала и конца ребра.
2. Вычисление длины ребра.
3. Запоминание оборудования при каждом из концов ребра.
4. Вычисление стоимости ребра (сумма стоимостей за длину ребра и всего оборудования на ребре).

```
class Connection:
    def __init__(
        self,
        node1: Node,
        node2: Node,
        node1_equip: Equipment | None = None,
        node2_equip: Equipment | None = None,
    ) -> None:
        store_attr()
        self.id = str(uuid())
        if self not in node1.connections:
            node1.connections.append(self)
        if self not in node2.connections:
            node2.connections.append(self)

    @property
    def length(self) -> float:
        return euclidean_distance(self.node1.coords, self.node2.coords)

    @property
    def cost(self) -> float:
        out = self.length * COST_PER_KM
```

```
        if self.node1_equip is not None:
            out += self.node1_equip.cost
        if self.node2_equip is not None:
            out += self.node2_equip.cost
        return out

    def get_opposite_node(self, caller: Node) -> Node:
        if caller.id == self.node1.id:
            return self.node2
        if caller.id == self.node2.id:
            return self.node1
        else:
            raise NoConnectionError(
                f"""ПЕРЕХОД НЕВОЗМОЖЕН. Произведена попытка
                перейти по ребру, соединяющему {self.node1.coords}
                и {self.node2.coords}, но caller {caller.coords}
                не является вершиной этого ребра."""
            )

    def __eq__(self, other: "Connection") -> bool:
        return self.id == other.id

    def __hash__(self) -> int:
        return hash(self.id)
```

1.1.3 `Equipment` — класс оборудования на соединениях

```
class Equipment:
    def __init__(self, cost: int) -> None:
        self._cost = cost

    @property
    def cost(self) -> int:
        return self._cost


class ManualSwitch(Equipment):
    def __init__(self) -> None:
        super().__init__(50000)


class AutoSwitch(Equipment):
    def __init__(self) -> None:
        super().__init__(250000)


class SubstationSwitch(Equipment):
    def __init__(self) -> None:
        super().__init__(150000)
```

1.1.4 `Network` — класс сети

Именно здесь, в этом классе, будут хранится данные о сети, а также происходить все расчёты. Из-за того, что класс получается большим, я разделю его методы по разным блокам кода, чтобы их можно было комментировать. Помечать каждый новый метод я буду с помощью `@patch_to(Network)`. При этом, `@patch_to(Network,`

`as_prop=True)` заменяет `@property`, а `@patch_to(Network, cls_method=True)` — `@classmethod`.

```
class Network:
    def __init__(self, nodes: List[Node], substation_node: SubstationNode) -> None:
        store_attr()
```

Как вы видите, мы отделяем вершину подстанции от массива остальных вершин, т.к. подстанция будет часто использоваться нами в будущем для подсчёта затрат.

Сейчас нам необходим способ инициализировать сеть без соединений, но с координатами нодов (вершин) так, как в условии. Дабы не загроможнать эту работу кодом, я пропущу реализацию фабричного метода `Network.init_from_default`. Скажу лишь, что это просто создание массива нодов с предопределёнными координатами и мощностями из условия. Вы также можете ознакомится с реализацией в репозитории `https://github.com/VEFrenovator/rn-2026/`.

Теперь от реализации самого интересного нас отделяет всего одни шаг — нам нужен способ посчитать стоимость нашей сети.

1.2 Рассчёт стоимости сети

Как я уже упомянул ранее, нам придётся жертвовать креативностью ради сохранения машинного рассчёта стоимости. Одна из главных потерь, с которой придётся смириться — мы не можем использовать ни разъединителей, ни реклоузеров. Дело в том, что у меня банально не хватает ни навыков, ни времени для валидного расчёта стоимости с ними. Я пытался (именно поэтому приведены реализации классов `Equipment, ManualSwitch, AutoSwitch, SubstationSwitch`), но ничего не получалось.

Второе упрощение — отказ от использования сетей с циклами и переход к деревьям. На первый взгляд может показаться, что это безрассудно, но на самом деле чуть позже я покажу математически, что без разъединителей и реклоузеров, в любой сети, отличной от дерева, нет никакого смысла.

Приняв такие упрощения во внимание, перейдём к самому рассчёту стоимости сети. Очевидно, стоимость (C) складывается из затрат на строительство (B) и ущерба (H)

$$C = B + H$$

1.2.1 Рассчёт затрат на строительство (B)

Затраты на строительство вычисляются достаточно тривиально: нам нужно лишь сложить сумму стоимостей постройки каждого ребра с стоимостью переключателей на ПС.

$$B = 150000 \cdot \alpha + \sum_{i=1}^{k} (l_i \cdot 500000)$$

(где α — количество исходящих из ПС рёбер; l_i — длина i-ого ребра.

$$B = 150000 \cdot \alpha + 500000 \cdot \sum_{i=1}^{k} l_i$$
$$B = 150000 \cdot \alpha + 500000 \cdot L_\Sigma$$

(где $L_\Sigma = \sum_{i=1}^{k} l_i$)

Из этой формулы непосредственно вытекает два факта:

1. Нам нужна единственная (магистральная) линия, выходящая из ПС (для минимизации α).
2. Нам нужно сделать суммарную длину рёбер минимальной из всех возможных (для минимизации L_Σ), т.е. нам необходимо дерево.

1.2.2 Рассчёт убытка от недоотпуска электроэнергии (H)

Формула из условия для рассчёта потерь от недоотпуска для одного потребителя с подставленными константами имеет вид:

$$H_i = 200 \cdot \lambda_\Sigma \cdot P_i + 50 \cdot D_\Sigma \cdot P_i$$

Общий убыток будет определяться как сумма убытков потребителей:

$$H = \sum_{i=1}^{n} (200 \cdot \lambda_\Sigma \cdot P_i + 50 \cdot D_\Sigma \cdot P_i)$$

При этом:

$\lambda_\Sigma = \sum_{i=1}^{k} (l_i \cdot \lambda) = \lambda \cdot \sum_{i=1}^{k} l_i = \lambda \cdot L_\Sigma$

$D_\Sigma = \sum_{i=1}^{k} (6 \cdot \lambda \cdot l_i) = 6 \cdot \lambda \cdot L_\Sigma$

$$H = \sum_{i=1}^{n} (200 \cdot \lambda \cdot L_\Sigma \cdot P_i + 50 \cdot 6 \cdot \lambda \cdot L_\Sigma \cdot P_i)$$

$$H = \sum_{i=1}^{n} (500 \cdot P_i \cdot \lambda \cdot L_\Sigma)$$

$$H = 500 \cdot P_\Sigma \cdot \lambda \cdot L_\Sigma$$

$$H = 500 \cdot 5120 \cdot 0.05 \cdot L_\Sigma$$

$$H = 128000 \cdot L_\Sigma$$

1.2.3 Рассчёт стоимости всей сети (C)

$$C = 150000 \cdot \alpha + 500000 \cdot L_\Sigma + 128000 \cdot L_\Sigma \tag{1}$$

Это и есть итоговая рассчётная формула, так что мы готовы добавить реализацию метода `formula_cost`

```python
@patch_to(Network, as_prop=True)
def length(self) -> float:
    self._length = 0.0
    self._seen_connections = set()
    self._get_surrounding_length(node=self.substation_node)
    return self._length


@patch_to(Network)
def _get_surrounding_length(self, node: Node) -> None:
    for c in node.connections:
        if c is not None and c not in self._seen_connections:
            self._length += c.length
            self._seen_connections.add(c)
            self._get_surrounding_length(node=c.get_opposite_node(caller=node))
```

```
@patch_to(Network, as_prop=True)
def formula_cost(self) -> float:
    substation_equip_count = 0
    for c in self.substation_node.connections:
        if c is not None:
            substation_equip_count += 1

    l = self.length
    return 150000 * substation_equip_count + 500000 * l + 128000 * l
```

Наконец, более ничего не отделяет нас от реализации интерактивности.

1.3 Интерактив — реализация и инструкция по использованию

Реализация функции `interactive` занимает почти 300 строк, потому я не буду здесь её приводить, тем более все реализации лежат в репозитории `https://github.com/VEFrenovator/rn-2026/`.

Говоря в целом, я создал микро-сайт, где реализована логика соединения и расстановки нод, и, самое главное, автоматического рассчёта стоимости сети.

Чтобы начать работу, перейдите по `https://rn-2026.onrender.com/`. Тем самым вы запустите инициализацию приложения. Если на сайт долго никто не заходил, запуск может занять до пяти минут или более. После загрузки вы увидете само приложение (см. 1а).

Описание интерфейса приложения (сверху вниз):

1. **Система координат (или график).** Самая высокоуровневая часть приложения. Если на окно навестись, то в его верхнем правом углу появится панель инструментов, которая включает в себя (слева направо):
   1.1. *Сохранить график как `PNG`.*
   1.2. *Увеличить до выбранного на графике прямоугольника.*
   1.3. *Перемещать график.*
   1.4. *Прямоугольное выделение.*
   1.5. *Произвольное выделение.*
   1.6. *Приблизить.*
   1.7. *Отдалить.*
   1.8. *Автоматическое приближение/отдаление.*
   1.9. *Вернуть исходное приблежение/отдаление.*
   1.10. *Перейти на сайт Plotly (с помощью которого создан этот график).*

   Более всего нас интересуют *прямоугольное* и *произвольное* выделения, т.к. именно ими можно выделять ноды, которые мы хотим соединить.
2. **Ползунок ввода x-координаты.**
3. **Ползунок ввода y-координаты.**
4. **Кнопка «Установить координаты подстанции».** Устанавливает координаты подстанции такими, какие они указаны в ползунках x и y координат.
5. **Кнопка «Установить координаты паечной точки».** Создаёт паечную точку по таким координатам, какие они указаны в ползунках x и y координат.
6. **Кнопка «Соединить выбранные точки».** Соединяет точки, выбранные на графике с помощью прямоугольного или произвольного выделения. За один раз можно соединять только 2 точки (иначе говоря, в выделении в момент нажатия на кнопку должны содержаться ровно 2 точки).

7. **Кнопка «Рассчитать затраты».** Выводит общую стоимость сети по формуле (1).

Для использования приложения сначала установите точку ПС, потом соединяйте ноды на ваше усмотрение и, если необходимо, добавляйте точки спайки. Затем рассчитайте стоимость полученной сети. Все шаги проиллюстрированы ниже.

Упомяну также, что на данный момент не реализована возможность удалять ненужные или случайно проставленные точки спайки и соединения, а также то, что координаты ПС можно установить лишь единожды. Если вы хотите перезапустить веб-приложение (вернутся к пустому графу, как это было изначально), просто перезагрузите страницу.

2 Стратегии оптимальных соединений

Теперь, имея такую большую вычислительную экосистему, мы готовы приступить к поиску стратегий оптимальных соединений. Я рассмотрю две сети (структуры) — *централизованную вокруг наибольших мощностей* и *древовидную с единичным корнем в* $(12.5, 0)$

2.1 Централизованная структура вокруг наибольших мощностей

Если внимательно посмотреть на список мощностей, можно понять, что в основном мощность не превышает $\sim$ 250 Вт. Однако, случаются выбросы до 300 Вт и 500 Вт. Более того, оказывается, что ноды по 500 Вт образуют подобие квадрата. Интуиция подсказывает, что можно группировать сеть вокруг них, подводя к ним магистральные линии от ПС и разводя их на множество маломощных потребителей. Так создаются *кластеры*.

Подмечу, что мощность работает здесь исключительно как интуитивное начинание, взятое из реального мира. Строго же говоря, мы можем подводить магистральные линии к нодам любой мощности, ведь в формуле (1) не фигурирует P. Опираясь на это, оптимизируем структуру: будем подводить магистраль не к 500-ваттным нодам, а к ближайшим нодам кластера.

См. 2. `Стоимость: 44’843’350.53452069`.

2.2 Древовидная структура с единичным корнем в $(12.5, 0)$

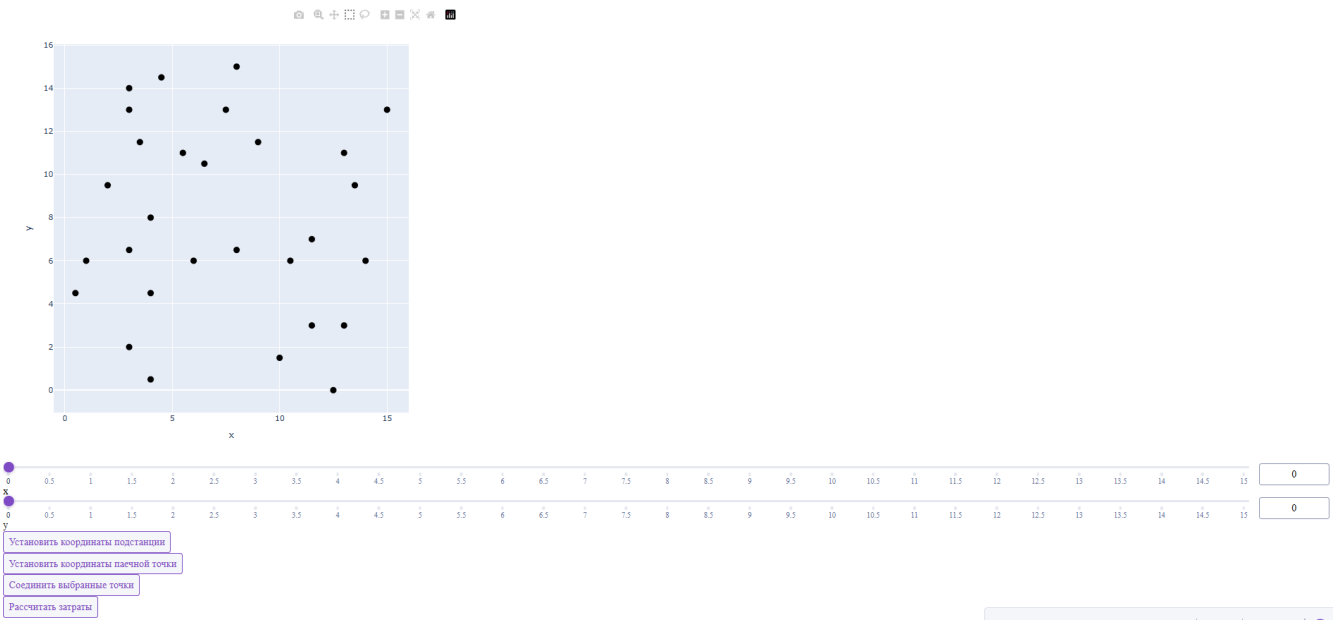
Недостаток предыдущей структуры, кажется, заключается в создании длинной «паутины», из-за чего суммарная длина рёбер увеличивается. Второй подход, реализованный мной, представляет из себя структуру, ветвящуюся по мере удаления от подстанции — корня. Так мы получаем систему, способную доставлять энергию до далёких от корня потребителей — листьев — без излишних изгибов, как это было при централизованной структуре.

В качестве координат для корня было два варианта: $(0, 4.5)$ и $(12.5, 0)$. Я выбрал второй, т.к. это ставит ПС непосредственно на 25-ю ноду, что снижает L_Σ на 0.5.

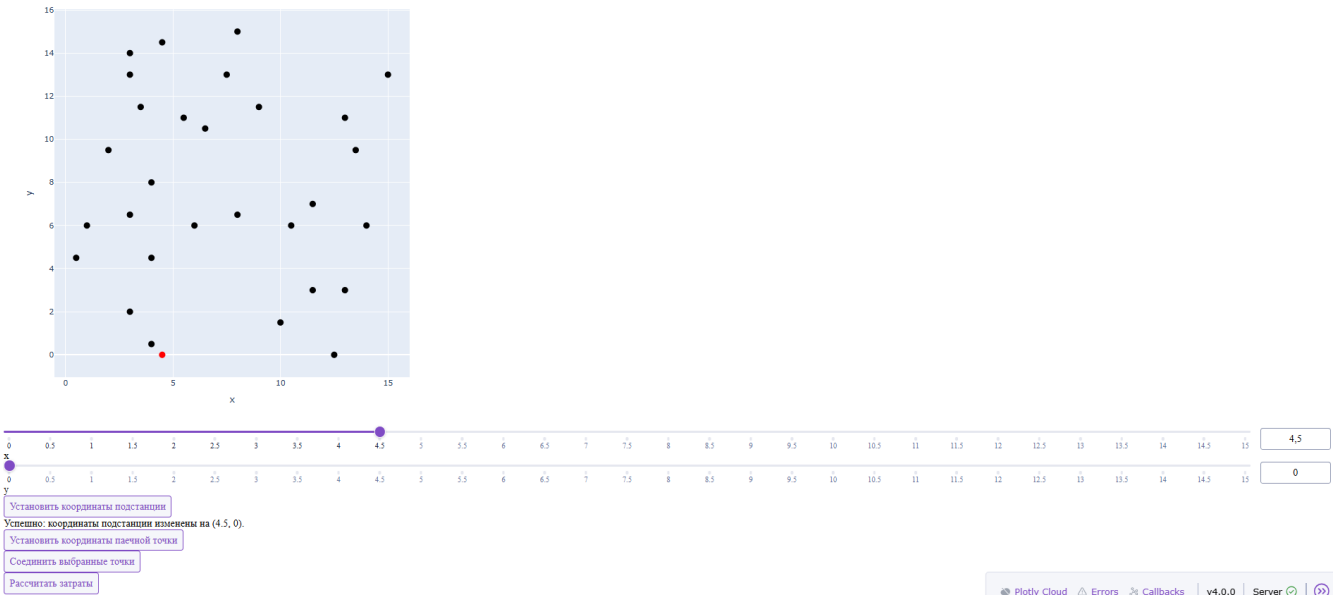
См. 3. `Стоимость: 41’911’581.056981685`.

3 Заключение

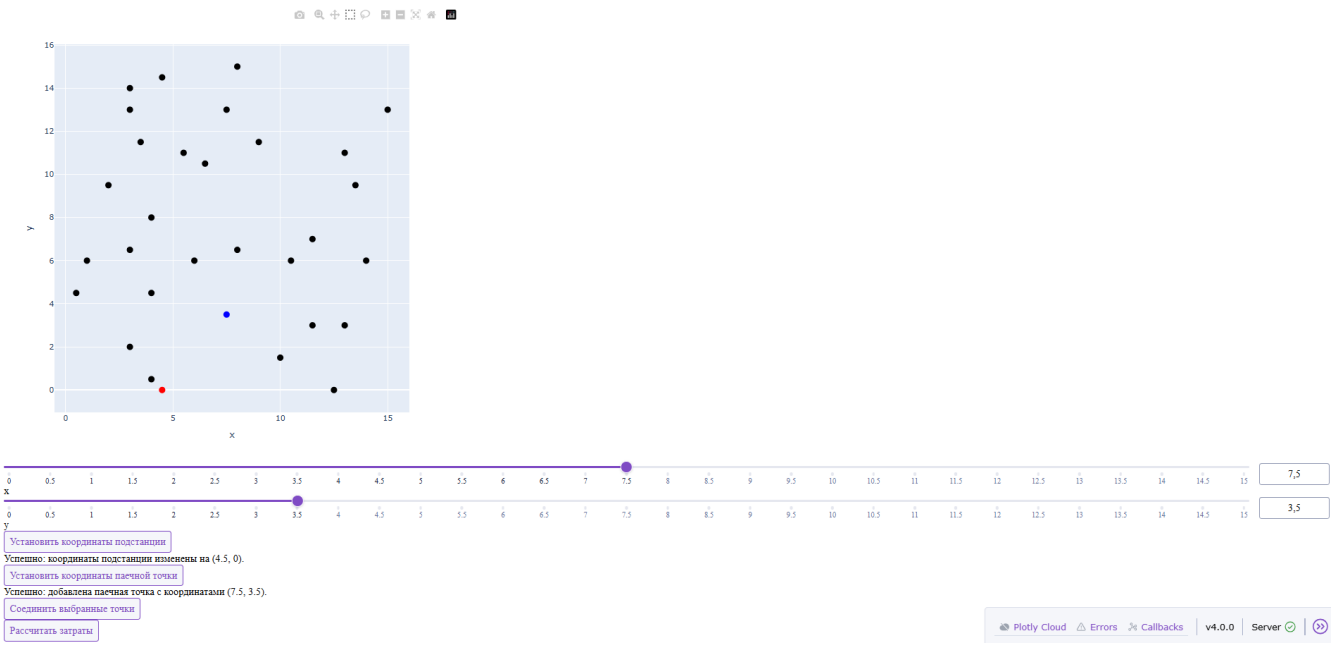
Подытоживая проделанную работу, можно говорить о достаточно полном выполнении кейсового задания олимпиады. Были реализованы классы и вся backend-экосистема обработки стоимости, а также frontend-сайт для интерактивного плэйграунда над задачей. Были применены некоторые упрощающие методы, позволившие, в свою очередь, реализовать полностью автоматический рассчёт стоимости сети. Интуитивные подходы вкупе с сделанными ранее математическими умозаключениями позволили реализовать две структуры оптимальных соединений. Лучший результат показала древовидная структура с единичным корнем в $(12.5, 0)$. Результат: $41'911'581.056981685$.



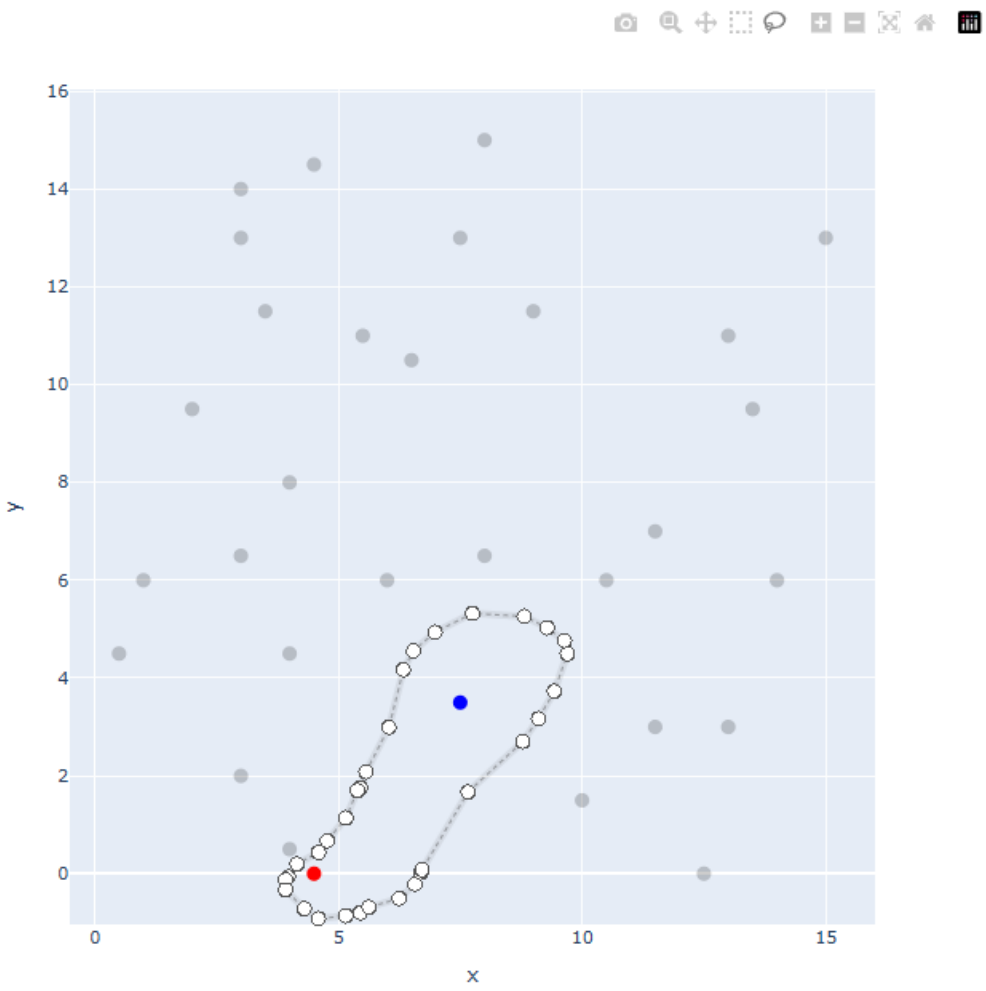
(a) Базовый вид приложения



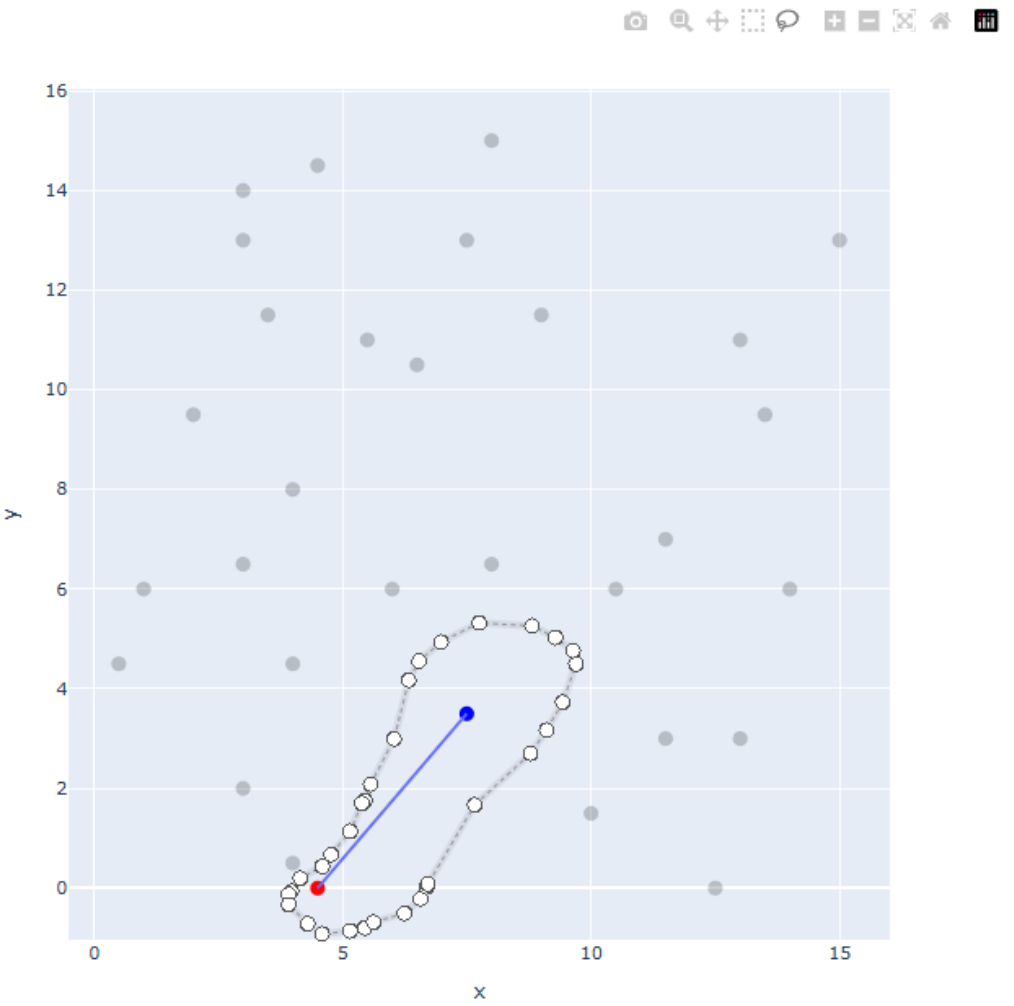
(b) После установки координат ПС



(c) После установки координат паечной точки

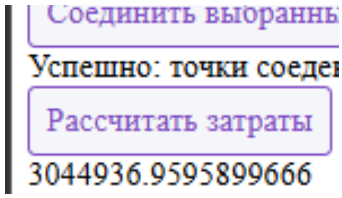


(d) После выделения (картинка приближена)





(e) После соединения двух нод (картинка приближена)



(f) После вычисления стоимости (картинка приближена)

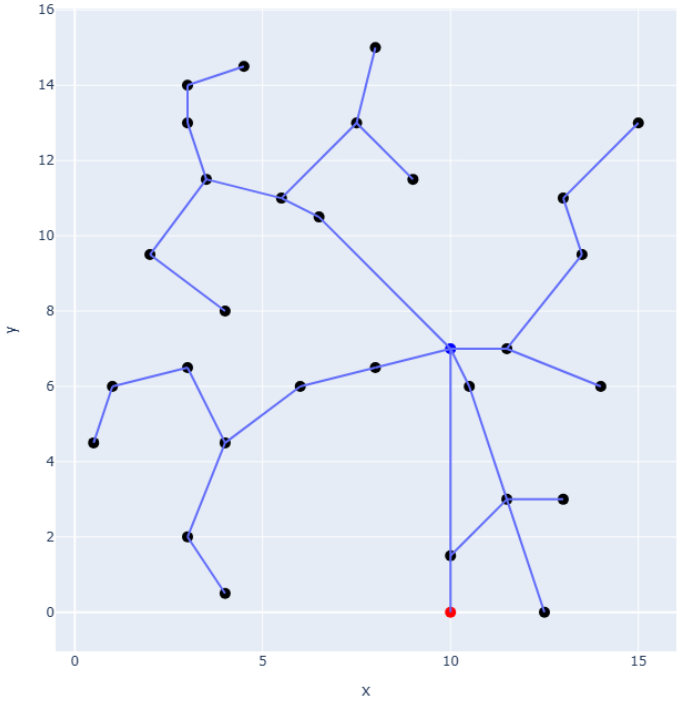


Рис. 2: Централизованная структура вокруг наибольших мощностей. `Стоимость: 44’843’350.53452069`

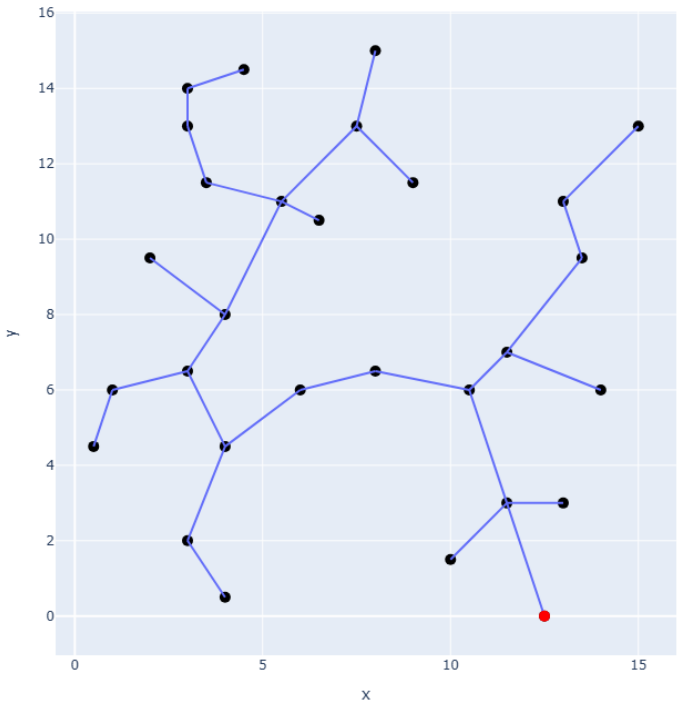


Рис. 3: Древовидная структура с единичным корнем в $(12.5, 0)$. `Стоимость: 41’911’581.056981685`